Program Output

```
Testing an adult membership...
Calling the showFees function with arguments 40 and 10.
The total charges are $400

Testing a senior citizen membership...
Calling the showFees function with arguments 30 and 10.
The total charges are $300

Testing a child membership...
Calling the showFees function with arguments 20 and 10.
The total charges are $200
```

As shown in Program 6-30, a driver can be used to thoroughly test a function. It can repeatedly call the function with different test values as arguments. When the function performs as desired, it can be placed into the actual program it will be part of.

Case Study: See High Adventure Travel Agency Part 1 Case Study on the Computer Science Portal at www.pearsonglobaleditions.com/gaddis.

Review Questions and Exercises

Short Answer

1. Why do local variables lose their values between calls to the function in which they are defined?
2. What is the difference between an argument and a parameter variable?
3. Where do you define parameter variables?
4. If you are writing a function that accepts an argument and you want to make sure the function cannot change the value of the argument, what do you do?
5. When a function accepts multiple arguments, does it matter in what order the arguments are passed?
6. How do you return a value from a function?
7. What is the advantage of breaking your application's code into several small procedures?
8. How would a `static` local variable be useful?
9. Give an example where passing an argument by reference would be useful.

Fill-in-the-Blank

10. In the _____ approach, a large problem is broken up into several smaller problems that can be easily solved.
11. The data type of the variable returned from a function is called _____.
12. Either a function's _____ or its _____ must precede all calls to the function.
13. The _____ function is automatically invoked when execution of a program starts.

14. Special variables that hold copies of function arguments are called _____.
15. When only a copy of an argument is passed to a function, it is said to be passed by _____.
16. A(n) _____ eliminates the need to place a function definition before all calls to the function.
17. A(n) _____ variable is defined inside a function and is not accessible outside the function.
18. _____ variables are defined outside all functions and are accessible to any function within their scope.
19. _____ variables provide an easy way to share large amounts of data among all the functions in a program.
20. When the _____ statement is encountered within a function, the control goes back to the statement that called the function.
21. If a function has a local variable with the same name as a global variable, only the _____ variable can be seen by the function.
22. _____ local variables retain their value between function calls.
23. A _____ is a named constant available to every function in a program.
24. _____ arguments are passed to parameters automatically if no argument is provided in the function call.
25. When a function uses a mixture of parameters with and without default arguments, the parameters with default arguments must be defined _____.
26. The value of a default argument must be a(n) _____.
27. When used as parameters, _____ variables allow a function to access the parameter's original argument.
28. Reference variables are defined like regular variables, except there is a(n) _____ in front of the name.
29. Only _____ can be passed as a reference to a function.
30. Functions that have the same name but different parameter lists are called _____
31. A _____ program is used to test whether a function is working properly.

Algorithm Workbench

32. Examine the following function header, and then write two different examples to call the function:

```
double absolute ( double number );
```

33. The following statement calls a function named average. The average function returns the average of all the arguments passed to the function. Write the function.

```
avg = average ( num1, num2, num3 );
```

34. A program contains the following function:

```
double velocity(double initial, double acceleration, double distance)
{
   return initial * initial + 2 * acceleration * distance;
}
```

Write a statement that passes the values 10.0, 50.5, and 100 to this function and assigns its return value to the variable `final_velocity`.

35. Write a function `isValid` that accepts an argument, grade. The function returns true if grade is within the range A through F; otherwise it returns false.

36. A program contains the following function:

```
void display(int arg1, double arg2, char arg3)
{
   cout  << "Here are the values: "
         << arg1 << " " << arg2 << " "
         << arg3 << endl;
}
```

Write a statement that calls the procedure and passes the following variables to it:

```
int age;
double income;
char initial;
```

37. Write a function named `getNumber` that uses a reference parameter variable to accept an integer argument. The function should prompt the user to enter a number in the range of 1 through 100. The input should be validated and stored in the parameter variable.

True or False

38. T F Functions should be given names that reflect their purpose.
39. T F Function headers are terminated with a semicolon.
40. T F A function may have more than one return statement.
41. T F If other functions are defined before `main`, the program still starts executing at function `main`.
42. T F When a function terminates, it always branches back to `main`, regardless of where it was called from.
43. T F Arguments are passed to the function parameters in the order they appear in the function call.
44. T F The scope of a parameter is limited to the function that uses it.
45. T F Changes to a function parameter always affect the original argument as well.
46. T F In a function prototype, the names of the parameter variables may be left out.
47. T F The lifetime of a local variable is throughout the execution of the program.
48. T F A local variable cannot have the same name as a global constant.
49. T F Static local variables are not destroyed when a function returns.
50. T F All static local variables are initialized to –1 by default.
51. T F Initialization of static local variables only happens once, regardless of how many times the function in which they are defined is called.
52. T F When a function with default arguments is called and an argument is left out, all arguments that come after it must be left out as well.

53. T F It is not possible for a function to have some parameters with default arguments and some without.
54. T F The `exit` function can only be called from `main`.
55. T F Stubs and drivers are tools that are used for testing and upgrading long programs that generally do not have any functions or sub-procedures in them.

Find the Errors

Each of the following functions has errors. Locate as many errors as you can.

56.
```
int product ( int num1; int num2; int num3 )
{
   return num1 * num2 * num3;
}
```

57.
```
double athird ( double number )
{
   int value = number / 3;

   return value;
}
```

58.
```
void area(int length = 30, int width)
{
   return length * width;
}
```

59.
```
void getValue(int value&)
{
   cout << "Enter a value: ";
   cin >> value&;
}
```

60. (*Overloaded functions*)
```
int getValue()
{
   int inputValue;
   cout << "Enter an integer: ";
   cin >> inputValue;
   return inputValue;
}

double getValue()
{
   double inputValue;
   cout << "Enter a floating-point number: ";
   cin >> inputValue;
   return inputValue;
}
```

Programming Challenges

1. **Markup**


VideoNote
Solving the Markup Problem

Write a program that asks the user to enter an item's wholesale cost and its markup percentage. It should then display the item's retail price. For example:

- If an item's wholesale cost is 5.00 and its markup percentage is 100 percent, then the item's retail price is 10.00.

- If an item's wholesale cost is 5.00 and its markup percentage is 50 percent, then the item's retail price is 7.50.

The program should have a function named `calculateRetail` that receives the wholesale cost and the markup percentage as arguments and returns the retail price of the item.

Input Validation: Do not accept negative values for either the wholesale cost of the item or the markup percentage.

2. Rectangle Area—Complete the Program

If you have downloaded this book's source code, you will find a partially written program named AreaRectangle.cpp in the Chapter 06 folder. Your job is to complete the program. When it is complete, the program will ask the user to enter the width and length of a rectangle, then display the rectangle's area. The program calls the following functions, which have not been written:

- `getLength`—This function should ask the user to enter the rectangle's length then return that value as a `double`.
- `getWidth`—This function should ask the user to enter the rectangle's width then return that value as a `double`.
- `getArea`—This function should accept the rectangle's length and width as arguments and return the rectangle's area. The area is calculated by multiplying the length by the width.
- `displayData`—This function should accept the rectangle's length, width, and area as arguments and display them in an appropriate message on the screen.

3. Winning Division

Write a program that determines which of a company's four divisions (Northeast, Southeast, Northwest, and Southwest) had the greatest sales for a quarter. It should include the following two functions, which are called by `main`:

- `double getSales()` is passed the name of a division. It asks the user for a division's quarterly sales figure, validates the input, then returns it. It should be called once for each division.
- `void findHighest()` is passed the four sales totals. It determines which is the largest and prints the name of the high-grossing division, along with its sales figure.

Input Validation: Do not accept dollar amounts less than $0.00.

4. Safest Driving Area

Write a program that determines which of five geographic regions within a major city (north, south, east, west, and central) had the fewest reported automobile accidents last year. It should have the following two functions, which are called by `main`:

- `int getNumAccidents()` is passed the name of a region. It asks the user for the number of automobile accidents reported in that region during the last year, validates the input, then returns it. It should be called once for each city region.
- `void findLowest()` is passed the five accident totals. It determines which is the smallest and prints the name of the region, along with its accident figure.

Input Validation: Do not accept an accident number that is less than 0.

5. Falling Distance

When an object is falling because of gravity, the following formula can be used to determine the distance the object falls in a specific time period:

$d = {}^1/_2 gt^2$

The variables in the formula are as follows: d is the distance in meters, g is 9.8, and t is the amount of time, in seconds, that the object has been falling.

Write a function named `fallingDistance` that accepts an object's falling time (in seconds) as an argument. The function should return the distance, in meters, that the object has fallen during that time interval. Write a program that demonstrates the function by calling it in a loop that passes the values 1 through 10 as arguments and displays the return value.

6. Throwing Distance

When throwing an object, the distance the object can reach is defined by the initial speed v and the angle α with which the object is thrown. The actual formula for the distance D is as follows:

$$D = \frac{v^2 . \sin(2\alpha)}{g}$$

The gravity constant g is defined as 9.81 m/s^2.

Write a function named `throwingDistance` that accepts the initial speed and the angle (in degrees) as arguments, and returns the throwing distance. Demonstrate the function by calling it in a program that asks the user to enter a value for the speed and print out all angles from 0 to 90° with steps of 5°.

7. Weight on Another Planet

The weight of any object on a planet depends on both the radius and the mass of the planet itself, but the formula becomes very easy if you start with the object's weight on Earth. For example,

- for Venus, multiply the weight on Earth with 0.905
- for Mars, multiply with 0.3787
- for Jupiter, multiply with 2.53

Write a function named `calculateWeight` that accepts the name of the planet and an object's weight on Earth as arguments, and returns the weight on that planet. Demonstrate the function by calling it in a program that asks the user to enter values for the planet and the weight.

8. Coin Toss

Write a function named `coinToss` that simulates the tossing of a coin. When you call the function, it should generate a random number in the range of 1 through 2. If the random number is 1, the function should display "heads." If the random number is 2, the function should display "tails." Demonstrate the function in a program that asks the user how many times the coin should be tossed, then simulates the tossing of the coin that number of times.

9. Present Value

Suppose you want to deposit a certain amount of money into a savings account and then leave it alone to draw interest for the next 10 years. At the end of 10 years, you would like to have $10,000 in the account. How much do you need to deposit today to

make that happen? You can use the following formula, which is known as the present value formula, to find out:

$$P = \frac{F}{(1 + r)^n}$$

The terms in the formula are as follows:

- P is the **present value**, or the amount that you need to deposit today.
- F is the **future value** that you want in the account. (In this case, F is $10,000.)
- r is the **annual interest rate**.
- n is the **number of years** that you plan to let the money sit in the account.

Write a program that has a function named `presentValue` that performs this calculation. The function should accept the future value, annual interest rate, and number of years as arguments. It should return the present value, which is the amount that you need to deposit today. Demonstrate the function in a program that lets the user experiment with different values for the formula's terms.

10. Future Value

Suppose you have a certain amount of money in a savings account that earns compound monthly interest, and you want to calculate the amount that you will have after a specific number of months. The formula, which is known as the future value formula, is

$$F = P \times (1 + i)^t$$

The terms in the formula are as follows:

- F is the **future value** of the account after the specified time period.
- P is the **present value** of the account.
- i is the **monthly interest rate**.
- t is the **number of months**.

Write a program that prompts the user to enter the account's present value, monthly interest rate, and the number of months that the money will be left in the account. The program should pass these values to a function named `futureValue` that returns the future value of the account, after the specified number of months. The program should display the account's future value.

11. Calorie Calculator

Write a function that calculates the number of "bad" calories in a recipe. In this assignment, calories are considered bad when coming from fat or sugar. It should use the following functions:

- `void getWeight()` should ask the user for a weight in pounds, store it in a reference parameter variable, and validate it. The function should be called for both the sugar and fat weights.
- `double calculateCaloriesFat(double weight)` accepts a weight as argument and returns the corresponding amount of calories by multiplying the weight with 4100.
- `double calculateCaloriesSugar(double weight)` accepts a weight as argument and returns the corresponding amount of calories by multiplying the weight with 936.
- `double calculateBadCalories()` must use the aforementioned functions to let the user input weights for fat and sugar and return the total of bad calories.

Input Validation: Do not accept weights less than zero.

12. Average Speed

A vehicle travels from city A to city B via city C. The average speed of the vehicle can be calculated by the following formula:

$$Average\ Speed = \frac{Distance\ from\ city\ A\ to\ city\ B + Distance\ from\ city\ B\ to\ city\ C}{Time\ taken\ to\ go\ from\ A\ to\ B + Time\ taken\ to\ go\ from\ B\ to\ C}$$

The time taken to cover any part of the journey can be calculated by the following formula:

$$Time\ taken\ from\ city\ X\ to\ city\ Y = \frac{Distance\ between\ X\ and\ Y}{Speed\ between\ X\ and\ Y}$$

Write a program that finds the average speed of the vehicle in its total journey from city A to city C using the above formulas. The program should ask the user to input the following:

- Distance between city A and city B
- Distance between city B and city C
- Speed of the vehicle while traveling from city A to city B
- Speed of the vehicle while traveling from city B to city C

The program should include a function named `average_speed` that accepts the four input values as parameters and returns the average speed. The program should then display the average speed.

13. Days Out

Write a program that calculates the average number of days a company's employees are absent. The program should have the following functions:

- A function called by `main` that asks the user for the number of employees in the company. This value should be returned as an `int`. (The function accepts no arguments.)
- A function called by `main` that accepts one argument: the number of employees in the company. The function should ask the user to enter the number of days each employee missed during the past year. The total of these days should be returned as an `int`.
- A function called by `main` that takes two arguments: the number of employees in the company and the total number of days absent for all employees during the year. The function should return, as a `double`, the average number of days absent. (This function does not perform screen output and does not ask the user for input.)

Input Validation: Do not accept a number less than 1 for the number of employees. Do not accept a negative number for the days any employee missed.

14. Days Since the Year Began

Write a function `getDaysInMonth` that accepts the number of the month as an argument: 1 for January, 2 for February, and so on. The function should return the number of days in that month in a non-leap year.

Demonstrate the function in a program that calculates the number of days since the beginning of the year. The user should enter the day (1–31), the month (1–12), and whether this is a leap year. Use the function `getDaysInMonth` to print out the number of days since the beginning of the year.

Input Validation: Months should be from 1 to 12, and days from 1 to 31.

15. Overloaded Hospital

Write a program that computes and displays the charges for a patient's hospital stay. First, the program should ask if the patient was admitted as an inpatient or an outpatient. If the patient was an inpatient, the following data should be entered:

- The number of days spent in the hospital
- The daily rate
- Hospital medication charges
- Charges for hospital services (lab tests, etc.)

The program should ask for the following data if the patient was an outpatient:

- Charges for hospital services (lab tests, etc.)
- Hospital medication charges

The program should use two overloaded functions to calculate the total charges. One of the functions should accept arguments for the inpatient data, while the other function accepts arguments for outpatient information. Both functions should return the total charges.

Input Validation: Do not accept negative numbers for any data.

16. Population

In a population, the birth rate is the percentage increase of the population due to births, and the death rate is the percentage decrease of the population due to deaths. Write a program that displays the size of a population for any number of years. The program should ask for the following data:

- The starting size of a population
- The annual birth rate
- The annual death rate
- The number of years to display

Write a function that calculates the size of the population for a year. The formula is

```
N = P + BP - DP
```

where `N` is the new population size, `P` is the previous population size, `B` is the birth rate, and `D` is the death rate.

Input Validation: Do not accept numbers less than 2 for the starting size. Do not accept negative numbers for birth rate or death rate. Do not accept numbers less than 1 for the number of years.

17. Transient Population

Modify Programming Challenge 16 to also consider the effect on population caused by people moving into or out of a geographic area. Given as input a starting population size, the annual birth rate, the annual death rate, the number of individuals who typically move into the area each year, and the number of individuals who typically leave the area each year, the program should project what the population will be `numYears` from now. You can either prompt the user to input a value for `numYears`, or you can set it within the program.

Input Validation: Do not accept numbers less than 2 for the starting size. Do not accept negative numbers for birth rate, death rate, arrivals, or departures.

18. Estimate Number of Containers

Bin packing is a typical algorithmic problem in computer science where one wants an optimal packing of boxes into as few containers as possible. In this assignment, we implement a very simple algorithm: we stack the boxes in a container in the same order as they are presented and start a new container as soon as the accumulated volume would exceed 100 ft^3. This means that we do not take into account the fact that boxes may not perfectly stack.

Write a function that takes the width, height, and depth of a box as arguments (in feet) and returns the volume of the box (`w*h*d`).

Use this function in a program that lets the user enter an unlimited number of boxes and calculates the number of containers needed. The user ends the input by entering 0.

Input Validation: The width, height, and depth should always be greater than 0.

19. Using Files—Hospital Report

Modify Programming Challenge 15 (Overloaded Hospital) to write the report it creates to a file.

20. Heating Energy

The heat capacity of a material is the amount of energy in joules it takes to heat up the material by 1°C or 1 K. For example, the heat capacity of water is approximately 4.2 J/(g K). This means that you need 4.2 joules (J) to heat up 1 gram (g) of water by 1 Kelvin (K).

Given that 1 lb equals approximately 453.59 grams and 1°F equals 5/9°C, write a function that takes as arguments the amount of water in lb and the desired temperature increase in °F. The function should return the amount of joules needed. Demonstrate the function in a program that asks the user to enter the necessary data.

Input Validation: Do not accept a negative value for the amount of water.

21. Heating Time

A typical electrical water heater consumes 1000 W: it generates 1000 J/s when switched on. Use this knowledge and the function of Programming Challenge 20 to write a function that takes as arguments the amount of water and the current temperature of the water. The function should return the time in seconds to heat the water to cooking temperature (212°F).

22. `isPrime` Function

A prime number is a number that is only evenly divisible by itself and 1. For example, the number 5 is prime because it can only be evenly divided by 1 and 5. The number 6, however, is not prime because it can be divided evenly by 1, 2, 3, and 6.

Write a function name `isPrime`, which takes an integer as an argument and returns `true` if the argument is a prime number, or `false` otherwise. Demonstrate the function in a complete program.

TIP: Recall that the % operator divides one number by another, and returns the remainder of the division. In an expression such as `num1 % num2`, the % operator will return 0 if `num1` is evenly divisible by `num2`.

23. Prime Number List

Use the `isPrime` function that you wrote in Programming Challenge 22 (`isPrime` function) in a program that stores a list of all the prime numbers from 1 through 100 in a file.

24. Rock, Paper, Scissors Game

Write a program that lets the user play the game of Rock, Paper, Scissors against the computer. The program should work as follows:

1. When the program begins, a random number in the range of 1 through 3 is generated. If the number is 1, then the computer has chosen rock. If the number is 2, then the computer has chosen paper. If the number is 3, then the computer has chosen scissors. (Don't display the computer's choice yet.)
2. The user enters his or her choice of "rock", "paper", or "scissors" at the keyboard. (You can use a menu if you prefer.)
3. The computer's choice is displayed.
4. A winner is selected according to the following rules:
   - If one player chooses rock and the other player chooses scissors, then rock wins. (The rock smashes the scissors.)
   - If one player chooses scissors and the other player chooses paper, then scissors wins. (Scissors cuts paper.)
   - If one player chooses paper and the other player chooses rock, then paper wins. (Paper wraps rock.)
   - If both players make the same choice, the game must be played again to determine the winner.

Be sure to divide the program into functions that perform each major task.

Group Project

25. Travel Expenses

This program should be designed and written by a team of students. Here are some suggestions:

- One student should design function `main`, which will call the other functions in the program. The remainder of the functions will be designed by other members of the team.
- The requirements of the program should be analyzed so each student is given about the same workload.
- The parameters and return types of each function should be decided in advance.
- Stubs and drivers should be used to test and debug the program.
- The program can be implemented as a multi-file program, or all the functions can be cut and pasted into the main file.

Here is the assignment: Write a program that calculates and displays the total travel expenses of a businessperson on a trip. The program should have functions that ask for and return the following:

- The total number of days spent on the trip
- The time of departure on the first day of the trip, and the time of arrival back home on the last day of the trip
- The amount of any round-trip airfare
- The amount of any car rentals
- Miles driven, if a private vehicle was used. Calculate the vehicle expense as $0.27 per mile driven
- Parking fees (The company allows up to $6 per day. Anything in excess of this must be paid by the employee.)
- Taxi fees, if a taxi was used anytime during the trip (The company allows up to $10 per day, for each day a taxi was used. Anything in excess of this must be paid by the employee.)
- Conference or seminar registration fees
- Hotel expenses (The company allows up to $90 per night for lodging. Anything in excess of this must be paid by the employee.)
- The amount of *each* meal eaten. On the first day of the trip, breakfast is allowed as an expense if the time of departure is before 7 a.m. Lunch is allowed if the time of departure is before 12 noon. Dinner is allowed on the first day if the time of departure is before 6 p.m. On the last day of the trip, breakfast is allowed if the time of arrival is after 8 a.m. Lunch is allowed if the time of arrival is after 1 p.m. Dinner is allowed on the last day if the time of arrival is after 7 p.m. The program should only ask for the amounts of allowable meals. (The company allows up to $9 for breakfast, $12 for lunch, and $16 for dinner. Anything in excess of this must be paid by the employee.)

The program should calculate and display the total expenses incurred by the businessperson, the total allowable expenses for the trip, the excess that must be reimbursed by the businessperson, if any, and the amount saved by the businessperson if the expenses were under the total allowed.

Input Validation: Do not accept negative numbers for any dollar amount or for miles driven in a private vehicle. Do not accept numbers less than 1 for the number of days. Only accept valid times for the time of departure and the time of arrival.